

Séance 2 – Arduino + AM2302 (DHT22)

Objectifs pédagogiques

À l'issue de cette séance, vous serez capable de :

- expliquer le fonctionnement interne du DHT22/AM2302 (mesure T° et HR, trame 40 bits, période minimale),
- réaliser le câblage correct avec résistance de pull-up entre VCC et DATA,
- décrire précisément l'utilisation de la bibliothèque Bonezegei_DHT22 (initialisation, lecture, récupération des valeurs),
- construire un flux de données au format JSON (JSON Lines) pour la suite Node-RED/Grafana,
- implémenter une temporisation non bloquante avec `millis()` (sans `delay()`) pour lire le capteur toutes les ~2 s.

1) Principe de fonctionnement du capteur AM2302 (DHT22)

Le DHT22 intègre :

- une thermistance pour la température ;
- un capteur capacitif pour l'humidité relative ;
- un microcontrôleur interne qui numérise ces mesures et les renvoie sur la ligne DATA.

Communication : protocole 1 fil propriétaire (similaire à One-Wire). La réponse contient 40 bits :

- 16 bits humidité, 16 bits température, 8 bits checksum.
- Le capteur impose une période minimale d'environ 2 secondes entre deux acquisitions.

2) Brochage et câblage (vue de face, texte lisible sur le capteur)

- Broches de gauche à droite : 1) VCC, 2) DATA, 3) NC, 4) GND.
- Alimentation : +3,3 V à +5 V (Arduino Nano → 5 V recommandé pour câbles >10 cm).
- Pull-up obligatoire entre VCC et DATA : 4,7 kΩ (à défaut : 2×2,7 kΩ en série ≈ 5,4 kΩ).

Raccordement type (Arduino Nano) :

- VCC (capteur) ↔ 5 V (Arduino)
- GND (capteur) ↔ GND (Arduino)
- DATA (capteur) ↔ entrée numérique (ex. D2)
- Pull-up : entre 5 V et DATA (proche du capteur si possible).

3) Bibliothèque Bonezegei_DHT22 – cycle d'utilisation

Installation : via le Gestionnaire de bibliothèques de l'IDE Arduino (rechercher « Bonezegei_DHT22 » et installer).

Cycle d'utilisation (ordre logique à suivre dans votre sketch) :

1. Déclarer la broche DATA du capteur et créer un objet de type Bonezegei_DHT22.

2. Dans la phase d'initialisation (setup), initialiser :

- la liaison série (afin d'observer les trames),
- la bibliothèque du capteur (appel à la fonction d'initialisation du capteur).

3. À chaque cycle de lecture (dans loop) :

- déclencher la lecture via la fonction prévue ;
- si la lecture réussit, récupérer la température (en °C par défaut, option °F possible) et l'humidité (%HR) ;
- si la lecture échoue, préparer une trame d'erreur ;
- émettre dans tous les cas une ligne JSON (succès/erreur) pour faciliter l'ingestion côté Raspberry Pi.

4) Format de sortie – JSON Lines (concept)

Le format JSON permet d'encoder les mesures dans une structure clé/valeur, lisible par des outils comme Node-RED. On émettra une ligne JSON par mesure (format « JSON Lines ») afin de faciliter l'ingestion en flux.

5) Pourquoi éviter delay() et utiliser millis() ?

Utiliser delay() bloque entièrement la boucle principale pendant la durée choisie. Cela empêche d'ajouter d'autres tâches en parallèle (lecture d'un autre capteur, clignotement d'une LED, communication série, etc.).

À l'inverse, millis() donne le temps écoulé (en millisecondes) depuis le démarrage du microcontrôleur. En mémorisant l'instant de la dernière lecture et en comparant régulièrement le temps courant, on déclenche la mesure uniquement lorsque l'intervalle minimal (≈ 2 s pour le DHT22) est écoulé, sans bloquer la boucle.

6) Comment utiliser millis() – méthode universelle (sans code)

Voici la démarche pas à pas à reproduire dans votre programme :

1. Définir une constante « période de lecture » en millisecondes (par ex. 2000 ms).
2. Déclarer une variable qui mémorise le « dernier instant de lecture » (type non signé 32 bits).
3. À chaque passage dans la boucle principale :
 - a) récupérer le « temps courant » (millisecondes depuis le démarrage),
 - b) calculer la différence « temps courant – dernier instant »,
 - c) si la différence est supérieure ou égale à la période :
 - mettre à jour le « dernier instant » avec le temps courant,
 - lancer la séquence d'acquisition du capteur (lecture + récupération valeurs),
 - construire et émettre la trame JSON correspondante (OK/erreur).

Remarques importantes :

- Cette méthode est robuste même lorsque millis() revient à 0 après $\sim 49,7$ jours

(débordement). Le test « temps courant – dernier instant $\geq$ période » reste valide si les variables sont bien de type non signé.

- Vous pouvez gérer plusieurs tâches périodiques en déclarant une paire « période/dernier instant » par tâche.
- Respecter la période minimale du DHT22 (≈ 2 s). Lire plus vite fera échouer les acquisitions.

7) Plan de construction du programme

Suivez ces étapes pour écrire votre programme :

1. Préparer le projet dans l'IDE Arduino et ajouter la bibliothèque Bonezegei_DHT22.
2. Choisir la broche DATA (ex. D2) et déclarer l'objet du capteur en conséquence.
3. Choisir la vitesse de la liaison série (115200 bauds recommandé) pour visualiser les trames.
4. Définir la période d'échantillonnage (2000 ms) et créer la variable « dernier instant de lecture ».
5. Dans l'initialisation :
 - démarrer la liaison série,
 - initialiser la communication avec le capteur (fonction d'initialisation de la bibliothèque).
6. Dans la boucle principale :
 - récupérer le temps courant (millisecondes),
 - si l'intervalle depuis la dernière lecture atteint la période :
 - mémoriser le nouvel instant,
 - déclencher la lecture du capteur,
 - si la lecture réussit :
 - récupérer température en °C et éventuellement en °F,
 - récupérer humidité en %,
 - préparer une ligne JSON « ok:true » avec les valeurs et l'horodatage,
 - si la lecture échoue :
 - préparer une ligne JSON « ok:false » (avec un champ « error » et l'horodatage),
 - envoyer la ligne JSON sur la liaison série (terminée par un retour à la ligne).
7. Vérifier dans le Moniteur Série qu'une ligne JSON est émise environ toutes les 2 s.

8) Conseils pour construire proprement la ligne JSON

- Émettre les clés dans un ordre constant : sensor $\rightarrow$ ok $\rightarrow$ temp_c $\rightarrow$ temp_f $\rightarrow$ hum $\rightarrow$ ts_ms.
- Limiter les décimales à 1 pour la température et l'humidité (utile pour la lisibilité et le stockage).
- Toujours terminer par un retour à la ligne pour faciliter le traitement en flux côté Node-RED.
- Utiliser des libellés simples (éviter les accents dans les clés JSON).

Validation & Dépannage

- Aucune ligne JSON : vérifier la vitesse du Moniteur Série et le port.
- Trames d'erreur répétées : vérifier la pull-up entre VCC et DATA, les masses communes, la broche sélectionnée.
- Valeurs incohérentes : éloigner le capteur des sources de chaleur, respecter la période minimale.
- L'intervalle réel n'est pas constant : vérifier la logique autour de millis et la mise à jour du « dernier instant ».